levelupedu.net | Coaching • Mentoring • Consulting

CLASS NOTES

PySpark — Resilient Distributed Datasets (RDD) — Session 02

Data Engineering Masterclass

Session Topics	groupByKey • mapValues • aggregateByKey • filter • distinct • 3 Real Use Cases
Pre-requisite	Basic Python — lists, lambda functions, tuples
Platform	Google Colab | PySpark 3.2.1 | Java 8
Course Module	Data Engineering Masterclass — PySpark Foundation
Trainer	Sateesh Pabbathi — Microsoft MVP | levelupedu.net

"Think of every concept like a real factory on the shop floor — not just code on a screen."

LevelUp

About Your Trainer

Sateesh Pabbathi

Microsoft MVP

22+ Years Experience

Empowering 5,000+ Data Engineers Across India

Sateesh Pabbathi is a Microsoft Most Valuable Professional (MVP) and the founder of LevelUp Cloud Trainings — one of India's most trusted data engineering education platforms. With over 22 years of hands-on industry experience, Sateesh has trained 5,000+ students and working professionals in cutting-edge data and cloud technologies.

His teaching philosophy is built on one powerful idea: make complex technology simple through real-world analogies. Whether it is comparing an RDD to a Zomato delivery team or explaining PySpark joins with Flipkart order data — Sateesh bridges the gap between theory and the actual Indian IT job market.

What You Get in This Course:

- Story-driven explanations using Zomato, Swiggy, HDFC Bank, TCS, Flipkart contexts
- Interview-ready content — every concept mapped to real job interview questions
- Hands-on labs on Google Colab, Databricks, and Microsoft Fabric
- Lifetime access to notes, assignments, and lab guides
- Direct mentoring support via WhatsApp broadcast and live sessions

Technologies Covered: Microsoft Fabric • Databricks • Azure Data Factory • PySpark • Python • SQL Server • GCP • AWS

🌐 **levelupedu.net** | 📱 **YouTube • LinkedIn • Instagram • WhatsApp**

"Your success in the data engineering field is my mission. Let us level up — together."

— Sateesh Pabbathi

Session 02 Overview

Session 01 Recap

Operation	What it Does
`parallelize / textFile / range`	3 ways to create RDD from data
`map()`	1 input => 1 output. Transforms each record
`flatMap()`	1 input => many outputs. Flattens nested data
`reduceByKey()`	Aggregates (K,V) pairs by key — memory efficient
`sortByKey() / sortBy()`	Sort by key or any derived value
`collect() / take()`	Actions that trigger execution and return data

New in Session 02

Operation	Purpose
`groupByKey()`	Group all values per key into a list. Use mapValues(list).
`aggregateByKey()`	Two-step aggregation: seqOp within partition, combOp across partitions.
`filter()`	Keep only elements where condition is True.
`distinct()`	Remove all duplicates. Returns unique elements only.
`mapValues()`	Transform the value of a (K,V) pair. Key unchanged.

Sateesh Says: Session 01 gave you the engine. Session 02 gives you the steering, brakes, and GPS. Together these 10+ operations are everything you need for any PySpark interview at TCS, Infosys, or Flipkart!

LevelUp
LevelUp

Section 2 — groupByKey() + mapValues()

What is groupByKey()?

groupByKey() collects all values that share the same key into a single iterable. It groups but does NOT aggregate — you decide what to do next.

Web Log Analogy: Think of a web server log. Each line says 'user1 visited /home'. groupByKey() collects ALL pages visited by user1 into one group: ('user1', ['/home', '/products', '/about']). Now you have the full browsing history per user.

Syntax: `rdd.groupByKey()  then  .mapValues(list)`

Example 1 — User Page Visits

```
# groupByKey — User Pages

data = [('user1','page1'),('user2','page1'),('user1','page2'),
        ('user2','page2'),('user3','page1')]
rdd = sparkContext.parallelize(data)

# groupByKey groups all pages per user
grouped_rdd = rdd.groupByKey()

# mapValues(list) converts lazy iterable to Python list
result = grouped_rdd.mapValues(list).collect()
print(result)
```

Output:

```
[('user1', ['page1', 'page2']),
  ('user2', ['page1', 'page2']),
  ('user3', ['page1'])]
```

Key Point: ALWAYS chain .mapValues(list) after groupByKey(). Without it, you get a lazy ResultIterable object — not a printable list.

Example 2 — Product Sales Grouping

```
# groupByKey — Product Sales

sales_data = [('product1',100),('product2',200),
              ('product1',300),('product3',400),('product2',500)]
rdd = sparkContext.parallelize(sales_data)
result = rdd.groupByKey().mapValues(list).collect()
print(result)
```

Output:

```
[('product1', [100, 300]),
```

```
 ('product2', [200, 500]),
 ('product3', [400])]
```

groupByKey() vs reduceByKey()

Feature	groupByKey() vs reduceByKey()
`Output value`	groupByKey = full list | reduceByKey = single aggregated value
`Memory usage`	groupByKey = HIGH (stores all) | reduceByKey = LOW (aggregates inline)
`Shuffle`	groupByKey shuffles raw values | reduceByKey shuffles pre-aggregated values
`Use when`	groupByKey: need the full list | reduceByKey: need sum, count, max

Sateesh Says: At Swiggy, groupByKey() collects all orders for a customer this month — you need the full list. reduceByKey() calculates the total bill — just one number. Know the difference and you ace the interview!

LevelUp
LevelUp

Section 3 — aggregateByKey() Transformation

What is aggregateByKey()?

aggregateByKey() is the most powerful aggregation in PySpark. It uses DIFFERENT logic within each partition (seqOp) and across partitions (combOp). This two-step approach allows computing multiple metrics — like sum AND count — in one pass.

> **School Exam Analogy:** 3 classrooms, same exam. Each room's teacher totals their students' scores locally (seqOp). The principal then combines all room totals (combOp). Divide total by count to get the overall average. That is aggregateByKey — partition-local first, then global combine.

The Three Parameters

Parameter	What it does
`zeroValue`	The starting value for each key's accumulator. Use (0,0) for (sum,count).
`seqOp`	Within one partition: how to combine a new value with the accumulator.
`combOp`	Across partitions: how to merge two accumulators from different machines.

Syntax: `rdd.aggregateByKey(zeroValue, seqOp, combOp).collect()`

Example 1 — Sum and Count per User

```
# aggregateByKey — Sum and Count

data = [('user1',10),('user2',20),('user1',30),('user2',40),('user3',50)]
rdd = spark.sparkContext.parallelize(data)

zero_value = (0, 0)  # (sum, count)

# seqOp: within partition — add value to sum, increment count
seqOp  = lambda acc, value: (acc[0] + value, acc[1] + 1)

# combOp: across partitions — add both sums and both counts
combOp = lambda acc1, acc2: (acc1[0] + acc2[0], acc1[1] + acc2[1])

result = rdd.aggregateByKey(zero_value, seqOp, combOp).collect()
print(result)
```

Output:

```
[('user1', (40, 2)), ('user2', (60, 2)), ('user3', (50, 1))]
```

LevelUp

user1: 10+30=40, count=2 | user2: 20+40=60, count=2 | user3: 50, count=1

Partition-by-Partition Trace — Alice & Bob

```
# Partition Trace

data = [('Alice',85),('Bob',70),('Alice',92),('Bob',88),('Alice',78),
('Bob',95)]
rdd = spark.sparkContext.parallelize(data, 2)

# PARTITION 1: Alice(85), Bob(70), Alice(92), Bob(88)
#   seqOp Alice: (0,0)->(85,1)->(177,2)
#   seqOp Bob:   (0,0)->(70,1)->(158,2)

# PARTITION 2: Alice(78), Bob(95)
#   seqOp Alice: (0,0)->(78,1)
#   seqOp Bob:   (0,0)->(95,1)

# COMBINE (combOp):
#   Alice: (177,2)+(78,1) = (255,3)  =>  avg = 255/3 = 85.0
#   Bob:   (158,2)+(95,1) = (253,3)  =>  avg = 253/3 = 84.3
```

Calculating Averages

```
# After aggregateByKey returns (sum, count), use map() for average
averages = rdd.aggregateByKey((0,0),
    lambda acc,val: (acc[0]+val, acc[1]+1),
    lambda a1,a2:  (a1[0]+a2[0], a1[1]+a2[1])) \
    .map(lambda x: (x[0], round(x[1][0]/x[1][1], 2)))
averages.collect()
# => [('Alice', 85.0), ('Bob', 84.33)]
```

⚠️ **Key Point:** aggregateByKey is the ONLY RDD way to compute averages per key in one pipeline. reduceByKey cannot track count AND sum simultaneously. This is a top interview question!

Sateesh Says: HDFC Bank uses this exact pattern: each branch (partition) totals its own transactions (seqOp). Head office combines all branch totals (combOp). That is the nightly reconciliation you see as 'Account Summary' in NetBanking!

Section 4 — filter() Transformation

What is filter()?

filter() creates a new RDD containing ONLY the elements where your condition returns True. It is the gatekeeper — only qualifying records pass through.

> **Quality Inspector Analogy:** A Maruti Suzuki factory quality inspector stands at the end of the assembly line. Cars that pass quality check go to the showroom (True = keep). Cars that fail go back for repair (False = discard). filter() is that quality inspector for your data.

Syntax: `filtered_rdd = rdd.filter(lambda x: condition)`

Example 1 — Even Numbers

```
# filter — Even Numbers

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
rdd = spark.sparkContext.parallelize(numbers)

# Keep only even numbers (remainder of x/2 == 0)
filtered_rdd = rdd.filter(lambda x: x % 2 == 0)
print(filtered_rdd.collect())
```

Output:

```
[2, 4, 6, 8, 10]
```

Example 2 — Abnormal Sensor Readings (Use Case 02 Preview)

```
# filter — Sensor Readings

sensor_data = [('M001',85,'2024-08-25'), ('M002',95,'2024-08-25'),
               ('M001',75,'2024-08-25'), ('M002',105,'2024-08-25'),
               ('M003',65,'2024-08-25'), ('M001',95,'2024-08-25')]
rdd = spark.sparkContext.parallelize(sensor_data)

# x[1] is the reading — keep only readings above 90
abnormal = rdd.filter(lambda x: x[1] > 90)
abnormal.collect()
```

Output:

```
[('M002', 95, '2024-08-25'),
  ('M002', 105, '2024-08-25'),
  ('M001', 95, '2024-08-25')]
```

Common filter() Patterns

Pattern	Example
`Greater than threshold`	rdd.filter(lambda x: x[1] > 90)
`String match`	rdd.filter(lambda x: x[0] == 'ACTIVE')
`Not None`	rdd.filter(lambda x: x is not None)
`Length condition`	rdd.filter(lambda x: len(x[1]) > 3)
`Multiple conditions`	rdd.filter(lambda x: x[1]>50 and x[2]=='Delhi')
`Stopword removal`	rdd.filter(lambda w: w not in stopwords)

Sateesh Says: At Zomato, filter() pulls only cancelled orders from a million records. At HDFC, it picks transactions above Rs 50,000 for RBI reporting. filter() is your most-used data cleaning operation!

LevelUp
LevelUp

Section 5 — distinct() Transformation

What is distinct()?

distinct() removes all duplicate elements from an RDD and returns only unique elements. It is like 'Remove Duplicates' in Excel — but across billions of distributed records.

Voter List Analogy: A government voter list must have each citizen only once. If someone registers twice, distinct() removes the duplicate entry. Or think of an expense report — if submitted twice by mistake, distinct() ensures it is counted only once.

Syntax: `distinct_rdd = rdd.distinct()`

Example 1 — Remove Duplicate Numbers

```
# distinct — Numbers

numbers1 = [1, 2, 3, 4, 5, 1, 2]
rdd1 = spark.sparkContext.parallelize(numbers1)

distinct_rdd1 = rdd1.distinct()
print(distinct_rdd1.collect())
```

Output:

```
[1, 2, 3, 4, 5]
```

Example 2 — Unique Page Visits (Use Case 01 Preview)

```
# distinct — Page Visits

# U001 visited /home twice — distinct() removes the duplicate
logs = [('U001','/home'),('U001','/products'),
        ('U001','/home'),('U002','/about'),('U002','/home')]

rdd = sparkContext.parallelize(logs)
unique = rdd.distinct()
unique.collect()
# U001's second /home visit is removed
```

Output:

```
[('U001', '/home'), ('U001', '/products'),
  ('U002', '/about'), ('U002', '/home')]
```

Key Point: distinct() triggers a shuffle across all partitions to find duplicates. On very large datasets this is expensive. Only use it when you genuinely need to remove duplicates.

Sateesh Says: Swiggy uses distinct() before counting unique customers per city. Without it, a customer who ordered 10 times would be counted 10 times. distinct() ensures accurate unique user counts — critical for business reporting!

LevelUp
LevelUp

Section 6 — Use Case 01: Website Log Analysis

Objective: Find TOP 2 users with most UNIQUE page visits

Dataset

```
# Use Case 01 — Dataset

logs = [
  ('U001','/home',     '2024-08-25 10:00:00'),
  ('U002','/about',    '2024-08-25 10:05:00'),
  ('U001','/products','2024-08-25 10:10:00'),
  ('U002','/home',     '2024-08-25 10:15:00'),
  ('U003','/products','2024-08-25 10:20:00'),
  ('U001','/home',     '2024-08-25 10:25:00')  # U001 visits /home again
]
rdd = spark.sparkContext.parallelize(logs)
```

Pipeline — Step by Step

Step	Operation & Purpose
Step 1	map() — Extract (user_id, page) only, drop timestamp
Step 2	distinct() — Remove duplicate (user, page) pairs
Step 3	groupByKey() + mapValues(list) — Collect unique pages per user
Step 4	sortBy(len) — Rank by number of unique pages (descending)
Step 5	take(2) — Get top 2 only

Full Solution

```
# Use Case 01 — Full Solution

# Step 1: Extract (user_id, page) — drop timestamp
users_pages_rdd = rdd.map(lambda x: (x[0], x[1]))

# Step 2: Remove duplicate page visits per user
distinct_rdd = users_pages_rdd.distinct()

# Step 3: Group all unique pages per user
grouped_rdd = distinct_rdd.groupByKey().mapValues(list)

# Step 4: Sort by number of unique pages (most active first)
sorted_rdd = grouped_rdd.sortBy(lambda x: len(x[1]), ascending=False)

# Step 5: Take only top 2 users
top_2 = sorted_rdd.take(2)
print(top_2)
```

LevelUp

Output:

```
[('U001', ['/home', '/products']),
  ('U002', ['/about', '/home'])]
```

Sateesh Says: This pipeline is what MakeMyTrip or BookMyShow uses to find their most engaged users. Replace page URLs with movie names, customer_ids with user_ids — and you have a recommendation engine foundation!

LevelUp
LevelUp

Section 7 — Use Case 02: Sensor Monitoring

Objective: Find machines with more than one abnormal reading (>90), sort by count

Dataset

```
# Use Case 02 — Dataset

sensor_data = [
  ('M001',85,'2024-08-25 10:00:00'),  # Normal
  ('M002',95,'2024-08-25 10:05:00'),  # Abnormal
  ('M001',75,'2024-08-25 10:10:00'),  # Normal
  ('M002',105,'2024-08-25 10:15:00'), # Abnormal
  ('M003',65,'2024-08-25 10:20:00'),  # Normal
  ('M001',95,'2024-08-25 10:25:00'),  # Abnormal
  ('M002',100,'2024-08-25 10:30:00')  # Abnormal
]
rdd = spark.sparkContext.parallelize(sensor_data)
```

Full Solution

```
# Use Case 02 — Full Solution

# Step 1: filter — keep only abnormal readings (> 90)
rdd2 = rdd.filter(lambda x: x[1] > 90)
# Keeps: M002/95, M002/105, M001/95, M002/100

# Step 2: map — convert to (machine_id, 1) for counting
rdd3 = rdd2.map(lambda x: (x[0], 1))

# Step 3: reduceByKey — count abnormal readings per machine
rdd4 = rdd3.reduceByKey(lambda x, y: x + y)

# Step 4: sortBy — most abnormal readings first
rdd5 = rdd4.sortBy(lambda x: x[1], ascending=False)
rdd5.collect()
```

Output:

```
[('M002', 3), ('M001', 1)]
```

M002 had 3 abnormal readings (95, 105, 100). M001 had 1 (95). M003 had none.

Sateesh Says: Tata Steel or Bosch uses this exact pipeline in IoT monitoring. Machines crossing safe thresholds get flagged and ranked by severity. You just built an industrial alert system with 4 lines of PySpark!

Section 8 — Use Case 03: E-commerce Transactions

Objective: Total spend per customer, sorted alphabetically by customer ID

Dataset

```
# Use Case 03 — Dataset

transactions = [
  ('C001','Item1',100), ('C002','Item2',150),
  ('C001','Item3',200), ('C002','Item1',300),
  ('C003','Item2',250), ('C003','Item3',100)
]
rdd = spark.sparkContext.parallelize(transactions)
```

Full Solution

```
# Use Case 03 — Full Solution

# Step 1: map — extract (customer_id, amount), drop item name
rdd2 = rdd.map(lambda x: (x[0], x[2]))

# Step 2: reduceByKey — sum all amounts per customer
rdd3 = rdd2.reduceByKey(lambda x, y: x + y)

# Step 3: sortByKey — sort alphabetically by customer ID
rdd4 = rdd3.sortByKey()
rdd4.collect()
```

Output:

```
[('C001', 300), ('C002', 450), ('C003', 350)]
```

C001: 100+200=300 | C002: 150+300=450 | C003: 250+100=350

Sateesh Says: This is Flipkart's monthly spend summary per customer. Add filter(lambda x: x[1] > 5000) at the end and you have their premium customer detection pipeline — simple operations, massive real-world impact!

Section 9 — Session 02 Recap & Cheat Sheet

What We Covered Today

1. groupByKey() — Groups values by key. Always pair with .mapValues(list).
2. mapValues() — Transforms only the value in a (K,V) pair. Key stays the same.
3. aggregateByKey() — Two-step aggregation: seqOp within partition, combOp across.
4. filter() — Keeps only elements where condition is True.
5. distinct() — Removes all duplicate elements. Returns unique records only.
6. Use Case 01 — Web Logs: map + distinct + groupByKey + sortBy + take.
7. Use Case 02 — Sensors: filter + map + reduceByKey + sortBy.
8. Use Case 03 — E-commerce: map + reduceByKey + sortByKey.

Interview Questions

- What is the difference between groupByKey() and reduceByKey()?
- Why do you always use .mapValues(list) after groupByKey()?
- What are the three parameters of aggregateByKey()? What does each do?
- How do you calculate average score per student using aggregateByKey()?
- What is the difference between filter() and distinct()?
- Why is groupByKey() memory-inefficient compared to reduceByKey()?

LevelUp
Coaching • Mentoring • Consulting